Adaptive Clustering Convexification Mapping-Planning System for UAVs in Unknown Environments

Yifei Zhang, Zheng Tan, Kang Liu, *Member, IEEE*, Yefeng Yang, Wenyu Yang, Shiyuan Wang, *Senior Member, IEEE*, and Chih-Yung Wen

***Abstract*—In complex environments, available moving space is often non-convex due to the presence of irregular obstacles, which complicates the mapping module's ability to represent the environment with geometric symbols. This complexity hampers the effectiveness of the planning module, leading to increased computational and storage demands. To address these challenges posed by non-convex spaces, we propose an adaptive clustering convexification mapping-planning system (ACCMPS) for unmanned aerial vehicles (UAVs) navigating in unknown environments. Our approach introduces an adaptive space-searching algorithm that transforms non-convex areas into relatively convex sub-spaces represented by clusters through purposeful sampling. ACCMPS utilizes a sample-friendly 3D grid map for local mapping and constructs a storage-efficient cluster-based weighted graph for global representation, linking convex sub-spaces represented by clusters. This integration streamlines the mapping and path planning processes, enabling direct initial path generation from the weighted graph, rather than relying on traditional algorithms that blindly explore non-convex spaces. To optimize paths, we employ a particle filter on the initial graph-derived path to handle non-convexity, followed by optimization into a smooth B-spline trajectory via cost function minimization within relatively convex regions. A series of real-world experiments conducted in both narrow and expansive environments demonstrates the advantages of ACCMPS in terms of computational efficiency, storage consumption, and path quality compared to existing state-of-the-art methods.**

***Note to Practitioners*—This paper proposes an Adaptive Clustering Convexification Mapping-Planning System (ACCMPS) for unmanned aerial vehicles (UAVs) navigating in unknown environments. The system enables UAVs to autonomously reach user-specified target points one by one while avoiding obstacles, even in completely unexplored spaces. By adaptively decomposing complex non-convex environments into relatively convex subspaces represented by clusters, ACCMPS significantly reduces computational and storage overhead compared to traditional methods that struggle with irregular obstacle layouts. The key advantage of ACCMPS lies in its efficient representation of the environment using a cluster-based weighted graph, which allows rapid global initial path generation without exhaustive exploration of local optimal regions in non-convex spaces. Path optimization is further refined through a particle filter and gradient-based optimization, ensuring safe and smooth B-spline-based trajectories. Real-world experiments in both confined and expansive environments validate its superior performance in computational speed, memory efficiency, and path quality. Potential applications include post-disaster rescue missions, where UAVs must efficiently explore chaotic, unmapped areas. For inspection tasks in industrial or agricultural settings, ACCMPS eliminates the need for pre-built maps, making it only requires a few rough waypoints as input to enable fully autonomous obstacle avoidance, significantly enhancing operational flexibility in unknown environments.**

***Index Terms*—Unmanned aerial vehicles (UAVs), robot navigation, mapping, path planning, trajectory planning.**

I. Introduction

UNMANNED aerial vehicles (UAVs) are playing an increasingly significant role in a wide range of applications, e.g., delivery, transportation, and rescue [1]. Among the core components of UAV navigation systems, mapping and planning are essential for constructing spatial representations of the environment and computing optimal trajectories to reach target destinations while avoiding obstacles. The core challenge addressed in this work is the pervasive non-convexity of navigable space in cluttered 3D environments. Non-convexity fundamentally complicates the entire navigation pipeline: it disrupts efficient mapping, enlarges the search space for planning, and creates optimization landscapes full of local minima. Existing systems are therefore forced into an unfavorable compromise, i.e., accurate mapping methods become too computationally heavy for real-time planning, whereas faster planning methods operate on coarse or redundant representations that reduce robustness in complex spaces. This persistent gap highlights the need for a unified mapping–planning framework that explicitly manages environmental non-convexity, enabling both computational efficiency and reliable path generation in unknown environments.

Received 15 August 2025; revised 21 January 2026; accepted 25 February 2026. Date of publication 5 March 2026; date of current version 12 March 2026. This article was recommended for publication by Associate Editor W. Zhang and Editor J. Yi upon evaluation of the reviewers' comments. This work was supported in part by the Research Centre of Unmanned Autonomous Systems, The Hong Kong Polytechnic University, under Grant P0046487; in part by the Research Centre for Low Altitude Economy, The Hong Kong Polytechnic University, under Grant P0058174; and in part by the Start-Up Fund for RAPs through the Strategic Hiring Scheme of The Hong Kong Polytechnic University under Grant P0059871. *(Corresponding author: Kang Liu.)*

Yifei Zhang, Zheng Tan, Kang Liu, Yefeng Yang, Wenyu Yang, and Chih-Yung Wen are with the Department of Aeronautical and Aviation Engineering, The Hong Kong Polytechnic University, Hong Kong (e-mail: yifei1999.zhang@connect.polyu.hk; zheng-uav.tan@connect.polyu.hk; kang1.liu@polyu.edu.hk; yefeng.yang@connect.polyu.hk; wenyu.yang@connect.polyu.hk; chihyung.wen@polyu.edu.hk).

Shiyuan Wang is with the College of Electronic and Information Engineering, Southwest University, Chongqing 400715, China (e-mail: wsy@swu.edu.cn).

Digital Object Identifier 10.1109/TASE.2026.3670813

1558-3783 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

This non-convexity first impairs efficient environmental representation, including the use of geometric symbols and uniform representation. The irregularity of obstacles and the non-convexity of the moving space complicate the representation of boundaries using geometric symbols, e.g., planes [2], [3] or line segments [4]. Additionally, directly representing the moving space with relatively convex hulls [5], [6] poses significant challenges. Compared to geometry-based maps, 3D grid maps [7], [8] and their variants, e.g., OctoMap [9], [10], are widely adopted for both local and global mapping. Although these methods offer a uniform representation of the environment that effectively accommodates irregular obstacles; however, the generated maps often contain significant redundant information, leading to increased storage requirements during the mapping process and extended computation time for the subsequent path planning module.

The representational shortcomings directly lead to inefficiencies in path planning. Although many algorithms have been proposed to enhance efficiency within 3D grid maps, these algorithms still face challenges arising from the non-convexity of the navigable space. For instance, sampling-based algorithms [11], [12], [13] encounter issues with blind sampling in non-navigable regions, making it difficult to identify the global optimal path. Artificial potential fields (APF)-based algorithms [14] can easily become trapped in local minima, while A*-based algorithms [15], [16], [17] often require considerable time to navigate around these traps. Furthermore, the exploration of space from previous iterations in the path planning module is rarely leveraged effectively, leading to time-consuming replanning operations within the global grid map.

Finally, non-convexity and discrete representations also undermine path optimization, which aims to enhance path quality by minimizing a cost function [18], [19]. This non-convexity results from the presence of obstacles that partition the environment into numerous subspaces, thereby complicating the structure of the cost function. Additionally, 3D grid maps, which are the most commonly used for mapping non-convex spaces, lack a continuous gradient of the distance to obstacles. This limitation hinders the effectiveness of gradient-based optimization methods in solving the path optimization problem.

To address the issues stemming from the non-convexity of the moving space, we propose an adaptive clustering convexification mapping-planning system (ACCMPS) for UAVs in unknown environments. To convert the non-convex moving space to relatively convex sub-spaces, the proposed adaptive space searching algorithm represents each convex sub-space through clusters generated by purposeful sampling. By doing so, this enhances the efficiency of both path planning and optimization in complex environments. In the proposed ACCMPS, the sample-friendly yet storage-consuming 3D grid map is employed as the local map to describe the environment surrounding the UAV. Meanwhile, the extensive global environment is represented by a storage-saving cluster-based weighted graph, which captures the linkages among the relatively convex sub-spaces represented by clusters. This approach bridges the traditionally decoupled mapping and path planning modules in conventional algorithms, enabling the direct generation of an initial path from the weighted graph. As a result, it avoids inefficient blind exploration in non-convex spaces and significantly reduces the computational time required by the subsequent path planning module. To mitigate the negative effects of non-convexity on path optimization, we employ a particle filter [20] based on the initial path derived from the graph to effectively address non-convex and gradient-free problems. Subsequently, we utilize an optimizer on this filtered path to generate a high-quality B-spline [21] trajectory by minimizing a cost function within relatively convex spaces. Finally, the robustness and effectiveness of the proposed ACCMPS is validated through the real-world experiments.

A. Related Works

Recent UAV navigation systems integrate mapping and planning modules to accelerate path planning compared to direct grid-based approaches. However, most struggle in narrow passages and complex environments. Inefficient mapping strategies and the loose coupling between mapping and planning often lead to high computational overhead during replanning.

Geometric representation-based methods model obstacles or free space using primitives such as convex hulls, polygons, or ellipsoids. For example, [22] generates safe flight corridors as overlapping convex polyhedra by first computing a piecewise collision-free path and then solving a QP with safety constraints. Similar works [23], [24], [25] apply convex decomposition for real-time motion planning. While effective in relatively simple environments, their performance degrades sharply in complex non-convex 3D spaces. Specially, many small convex hulls improve resolution but increase computational overhead, whereas fewer large hulls reduce accuracy and hinder navigation in tight spaces. As a result, such techniques are more suitable for 2D UGV applications, where convex decomposition is cheaper and fine grids remain feasible, than for 3D UAV environments where higher-dimensional representations impose much greater computational costs.

Sampling-based methods form another major category. PRM-based approaches [26], [27] sample feasible regions and build graphs for optimal path search. These methods are effective in open areas but struggle at narrow passages where dense sampling is required to maintain connectivity. Moreover, large portions of sampling effort are wasted in enclosed or infeasible regions, leading to oversized graphs, long planning times, and high memory load.

Learning-based approaches, particularly deep reinforcement learning (DRL), can learn reactive navigation policies without explicit mapping and work well in relatively simple spaces [28], [29], [30]. Nevertheless, these approaches often generalize poorly to tight or highly constrained environments due to limited spatial reasoning and face additional challenges, including high training cost, sim-to-real gaps, and limited interpretability. The aforementioned factors hinder their reliability and deployment in safety-critical UAV missions.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

A related research direction is risk-bounded motion planning under uncertainty, where nonconvex static or dynamic obstacles are modeled by probabilistic representations. Recent work [31] formulates continuous-time risk-bounded trajectory planning by defining collision risk as the probability of intersecting uncertain obstacles and introducing risk contours to transform chance-constrained problems into deterministic optimization forms. In [32], an optimization-based approach converts stochastic dynamics and chance constraints into deterministic moment-based formulations, enabling the handling of arbitrary uncertainty distributions. Meanwhile, the learning-based method [33] can complete this task by encoding uncertain obstacle point clouds into latent spaces to support informed sampling and probabilistic risk verification. While effective for uncertainty-aware navigation, these methods typically need explicit probabilistic obstacle models and nontrivial risk evaluation, which can be challenging for real-time UAV operation in fully unknown and complex nonconvex environments.

B. Contributions

The contributions of this paper are summarized as follows:

1) The ACCMPS tackles the challenges of non-convex moving spaces by transforming them into relatively convex sub-spaces through adaptive clustering. This approach significantly reduces storage consumption by simplifying the environmental representation, thereby facilitating more efficient path planning.
2) The proposed system integrates the mapping and planning modules by utilizing a cluster-based weighted graph to represent the global environment. This integration eliminates blind exploration in non-convex spaces and significantly reduces the computational time required for path planning.
3) The proposed two-stage path optimization module generates a high-quality B-spline trajectory that balances smoothness, distance to obstacles, and path length within the non-convex space. This comprehensive approach enhances the overall effectiveness of path planning in complex environments.

II. Overview

ACCMPS is a mapping–planning system that can be embedded in a UAV navigation framework for complex environments. The objective of ACCMPS is to continuously generate a collision-free and dynamically feasible trajectory that guides the UAV toward the target point while maintaining a safe distance from obstacles. The inputs for ACCMPS consist of a depth map, represented as $\boldsymbol{D} \in \mathbb{R}_{\geq 0}^{a\times b}$, in which pixel intensities encode the distance from the depth camera to the observed scene; the pose of the depth camera in the world coordinate frame, given by the homogeneous transformation matrix $\mathbf{T}_{wd} \in \mathrm{SE}(3)$ with rotation $R_{wd} \in \mathrm{SO}(3)$ and translation $t_{wd} \in \mathbb{R}^3$; and the target point $P_{tar} \in \mathbb{R}^3$. The output is a cubic B-spline trajectory $\boldsymbol{B}$, represented by control points $\boldsymbol{Q} = \{\boldsymbol{Q}_1, \boldsymbol{Q}_2, \ldots, \boldsymbol{Q}_{N_c}\}$ and knot vector $U = \{0, 0, 0, 0, u_0, u_1, \ldots, u_{N_c-3}, u_{N_c-3}, u_{N_c-3}, u_{N_c-3}\}$, where $\boldsymbol{Q}_i \in \mathbb{R}^3$ and $u_m \in \mathbb{R}$.

The underlying problem modeled by ACCMPS is to compute, at each time step k, a B-spline $\boldsymbol{B}_k(t)$ that minimizes the deviation from the desired target direction while satisfying two essential constraints: (1) the trajectory is required to maintain a prescribed safety margin from all occupied grids in the environment map, and (2) the trajectory needs to meet respect the UAV's dynamic limits on velocity, acceleration, and angular velocity. To enable this, ACCMPS maintains both a local and a global map, $\boldsymbol{G} = \{\boldsymbol{G}^l, \boldsymbol{G}^g\}$, and updates them online using incoming depth measurements.

The general model of ACCMPS is formulated as:

$$\begin{aligned} &\boldsymbol{G}_k = \textbf{Mapping}(\boldsymbol{G}_{k-1}, \boldsymbol{D}_k, \mathbf{T}_{wd,k}),\\ &\boldsymbol{B}_k(t) = \textbf{Planning}(\boldsymbol{G}_k, \mathbf{T}_{wd,k}),\\ &\text{s.t. } \textbf{DisttoObj}(\boldsymbol{G}^l, \boldsymbol{B}_k(t)),\\ &\textbf{PathContinuity}(\boldsymbol{B}_k(t)), \end{aligned} \tag{1}$$

where k and $k-1$ denote the current and previous time steps. In (1), **Mapping**() fuses sequential depth images to incrementally update the environment representation; **Planning**() generates an initial collision-free path and refines it using an optimization model; **DisttoObj**() ensures that the trajectory keeps a safe distance to obstacle grids; and **PathContinuity**() enforces the smoothness and dynamic feasibility required for UAV control. This formulation captures the core problem studied in this work: real-time mapping and optimization-based trajectory generation for safe, smooth, and target-directed UAV navigation in unknown environments.

The diagram of the entire UAV navigation system is shown in Fig. 1. The UAV navigation system integrates three functional modules as localization, mapping, and path planning. The localization module employs VINS-Fusion [34] to estimate the UAV's pose in real time by fusing data from an RGB camera and an IMU, providing robust and accurate state estimation. The mapping module processes depth camera point clouds to construct an efficient environmental representation, combining a 3D grid map for local obstacle avoidance and an adaptive cluster-based weighted graph map for global path planning. This dual-layered mapping strategy retains only relevant spatial information, thereby reducing storage consumption while maintaining planning reliability. For the trajectory generation, the planning module first searches the weighted graph to produce an initial path, then refines it in non-convex environments using a particle filter. Finally, the path is optimized into a smooth B-spline curve that adheres to the UAV's kinematic constraints. The resulting trajectory is tracked by the flight controller via a feedforward PID, enabling autonomous navigation in complex environments.

III. Mapping

As shown in Fig. 2, the mapping module uses the real environment perceived by sensors to build a virtual environment containing useful information for the path planning module. To reduce computational consumption, the proposed 3D grid map is used to reflect the environment near the UAV. Although grid maps typically consume more storage,

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

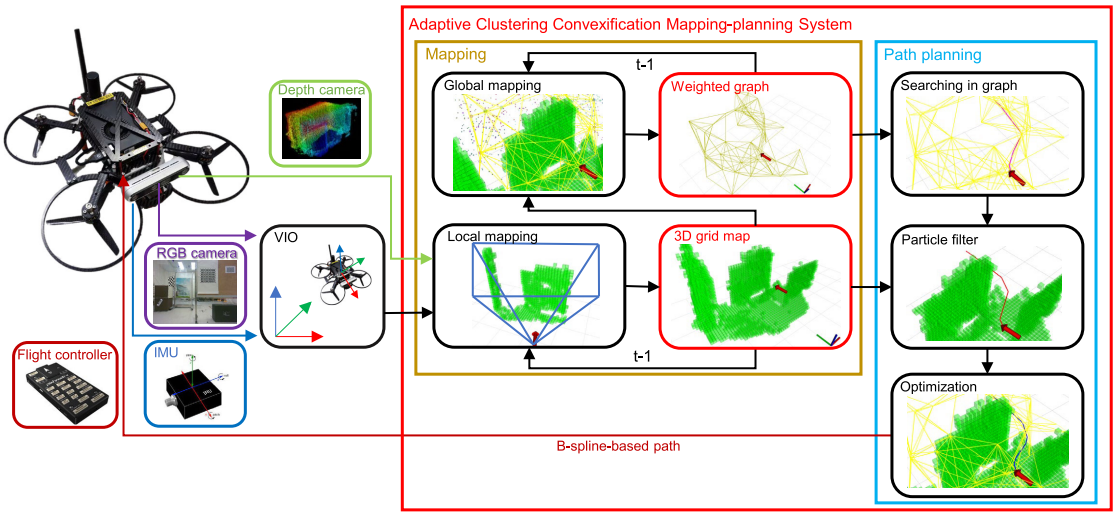


Fig. 1. Diagram of the entire UAV navigation system. RGB images and depth maps are captured by Intel D455 and ICM-42688P IMU is used to achieve acceleration and angular velocity.

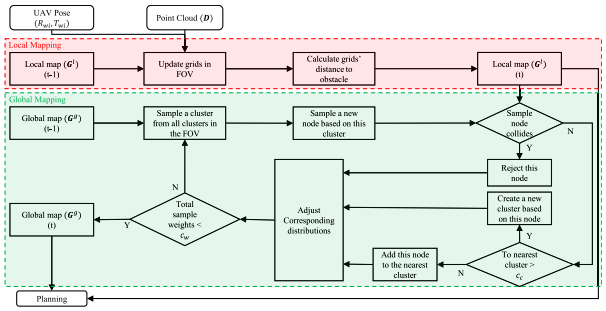


Fig. 2. Framework of the mapping module.

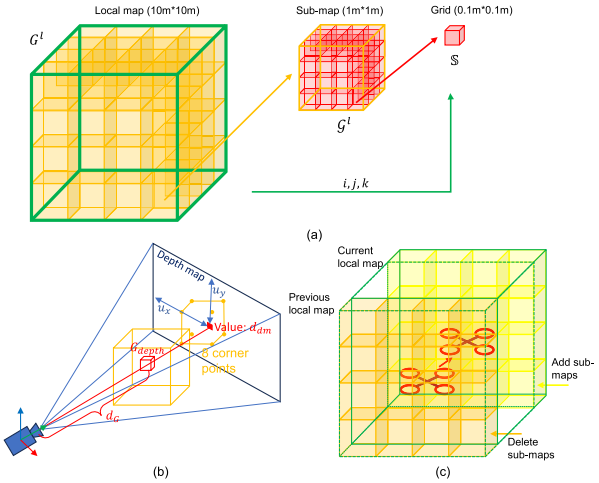


Fig. 3. Diagram of local mapping. (a) Sub-map strategy is used to increase the efficiency of the mapping module. (b) Process of mapping. Projecting points in the grid to the depth map's pixels, and checking the distance of points' distances to UAV and the depth data on pixels. (c) Adding and Deleting one layer of sub-maps to keep the UAV at the center of the local map.

it is more friendly for sampling-based algorithms to search the moving space. In the UAV navigation task, this issue is negligible because the coverage area of the local map is significantly smaller than that of the global map, particularly in extensive environments. Additionally, an adaptive clustering weighted graph is utilized to represent the connections among all moving spaces indicated by clusters in the expansive global map, resulting in significant storage savings.

A. Local Mapping

The local mapping sub-module is to reflect the environment near the UAV by the proposed 3D grid map $\boldsymbol{G}^l$ from the local map in the previous time $\boldsymbol{G}^l$, depth map $\boldsymbol{D}$, and pose of the UAV in the world coordinate system R_{wi} and t_{wi}. In ACCMPS, the sub-map strategy shown in Fig. 3 (a) is adopted to store the local map, which can be described as

$$\begin{aligned} &\boldsymbol{G}^l \in \mathcal{G}^{l\ M\times N\times P}, \\ &\mathcal{G}^l \in \mathbb{S}^{\ m\times n\times p}, \\ &\mathbb{S} = \{(d_{obj}, b_{ob}) | d_{obj} \in \mathbb{R}^+, b_{ob} \in \mathbb{B}\}, \end{aligned} \tag{2}$$

where M, N, and P are the number of sub-maps in length, width, and height directions, respectively; m, n, and p are the grids in length, width, and height diections of each sub-map, respectively; d_{obj} is the distance from this grid to the nearest obstacle; b_{ob} is a bool value to store whether this grid is observing by the depth camera.

The process of updating the local map is outlined in Algorithm 1, with the corresponding diagram illustrated in Fig. 3 (b). The sub-functions are described as follows.

- **toDepthSystem**$(\boldsymbol{G}^l_{i,j,k}, R_{wd}, t_{wd})$: Transform the center of the grid $\boldsymbol{G}^l_{i,j,k}$ from the world coordinate system to that of the depth camera by R_{wd} and t_{wd}.
- **toPixelSystem**$(\boldsymbol{G}_{depth}, K)$: Transform $\boldsymbol{G}_{depth}$ to the pixel coordinate system in the depth camera.
- **notinDepthCameraSight**(ux, uy): Return whether this is out of the maximum range of the depth map.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 UpdatingLocalMap($\boldsymbol{D}, R_{wd}, t_{wd}, \boldsymbol{G}^l$)

Notation: K is the intrinsic matrix of the depth camera;
for $\mathcal{G}^l_{I,J,K}$ *in* $\boldsymbol{G}^l$ **do**
  $\boldsymbol{G}_{depth} \leftarrow$ **CornerstoDepthSystem**($\boldsymbol{G}^l_{i,j,k}, R_{wd}, t_{wd}$);
  $Ux, Uy \leftarrow$ **CornerstoPixelSystem**($\boldsymbol{G}_{depth}, K$);
  if ***AllnotinDepthCameraSight***(Ux, Uy) **then**
    continue;
  end
  for $\mathcal{G}^l_{i,j,k}$ *in* $\mathcal{G}^l_{I,J,K}$ **do**
    $\mathcal{G}_{depth} \leftarrow$ **toDepthSystem**($\mathcal{G}^l_{i,j,k}, R_{wd}, t_{wd}$);
    $ux, uy \leftarrow$ **toPixelSystem**($\mathcal{G}_{depth}, K$);
    if ***notinDepthCameraSight***(ux, uy) **then**
      $\mathcal{G}^l_{i,j,k}.b_{ob} \leftarrow 0$;
      continue;
    end
    $d_{dm} \leftarrow \boldsymbol{D}_{ux,uy}$;
    $d_G \leftarrow |G_{depth}|$;
    //By the following algorithm, the moving object and the object behind it can also be reflected correctly ;
    if $d_G - d_{dm} < gridSize$ **then**
      $\mathcal{G}^l_{i,j,k}.b_{ob} \leftarrow 1$;
    end
    else if $-gridSize < d_G - d_{dm} < gridSize$ **then**
      $\mathcal{G}^l_{i,j,k}.d_{obj} \leftarrow 0$; $\mathcal{G}^l_{i,j,k}.b_{ob} \leftarrow 1$;
    end
    else if $-2gridSize < d_G - d_{dm} < -gridSize$ **then**
      $\mathcal{G}^l_{i,j,k}.b_{ob} \leftarrow 0$;
    end
  end
end
UpdatingDisttoObj($\boldsymbol{G}^l$);
if $|t_{wd}\boldsymbol{G}^l.center| > subMapSize$ **then**
  MovingMap($\boldsymbol{G}^l$);
end
return($\boldsymbol{G}^l$);

- **UpdatingDisttoObj**($\boldsymbol{G}$): Change the minimum distance to obstacle d_{obj} of all grids in $\boldsymbol{G}$ by the wavefront propagation algorithm [35], which is an iterative algorithm used to compute the shortest distance from a set of source points, e.g., to all other points in a grid or spatial domain by propagating a "wave" outward from the sources.
  MovingMap($\boldsymbol{G}^l$): Make the UAV at the center of the local map by deleting one layer of sub-map in the direction opposite the UAV's moving direction and then adding a new layer of sub-map in the direction of the UAV's moving.

In the ACCMPS, the entire local map is defined as a cubic region with dimensions of 10m × 10m × 10m, while each sub-map is set to 1m × 1m × 1m, and the grid resolution is 0.1m. This hierarchical structure offers two significant advantages in terms of computational efficiency. In the subsequent analysis, we define n as the number of grids along each edge of the entire local map, N_1 as the number of sub-maps along each edge of the local map, and N_2 as the number of grids along each edge of a sub-map.

In the following, we introduce the process for maintaining the local map. First, due to the high resolution of depth images (typically higher than 640 × 480), it is time-consuming to transform every pixel into world-frame coordinates and assign it directly to the corresponding grids, as is commonly done when processing LiDAR point clouds. Moreover, this pixel-wise method is computationally expensive when handling moving objects. To address these issues, we adopt a reverse procedure: instead of iterating over all pixels, we traverse every grid within the UAV's field of view and transform the grid center from the world frame to the pixel frame of the depth image. By comparing the distance from each grid to the depth camera with the corresponding value in the depth map, we can efficiently determine whether the grid lies in front of, near, or behind an obstacle, and update the state $\mathbb{S}$ of the grid accordingly. With this approach, grids in front of an obstacle are marked as free space, grids near the obstacle are marked as occupied, and grids behind the obstacle retain their previous state. As a result, moving objects can be correctly updated in the map, while static obstacles that become temporarily occluded by moving objects remain preserved.

Although this method avoids iterating through every pixel of the depth map, it still requires scanning the entire local map, which remains computationally expensive. To mitigate this problem, we introduce a double-layer sub-map strategy. If all eight corners of a sub-map lie outside the UAV's field of view, the entire sub-map can be skipped, significantly reducing computation time, as illustrated in Fig. 3(b).

As illustrated in Fig. 3(b), the sub-map strategy eliminates the need to perform visibility checks for every individual grid. Without sub-maps, all grids would require evaluation, resulting in a time complexity of $O(n^3) = 100^3$. By contrast, the proposed method only requires checking the visibility of the eight corner points of each sub-map. If all corners are outside the camera's field of view, all grids within that sub-map can be skipped. This reduces the time complexity to $O(N_1^3 + k \cdot N_2^3) = 10^3 + k \cdot 10^3$, where k represents the number of occupied sub-maps. Since most sub-maps are empty (i.e., $k \ll N_1^3$), the computational savings are substantial.

As the UAV travels, the local map must be dynamically updated to keep the UAV near its center. Instead of directly shifting all grids, which is inefficient, the sub-map strategy enables a more effective approach. Specifically, we remove one layer of sub-maps in the direction opposite to the UAV's movement and append a new layer in the direction of motion, as shown in Fig. 3(c). The sub-map strategy allows efficient map updates by deleting and creating entire layers of sub-maps with a time complexity of $O(2 \cdot N_1^2) = 2 \times 10^2$. In a naive implementation without sub-maps, each affected grid would need individual processing, leading to a higher time complexity of $O(2N_1 \cdot n^2) = 2 \times 10 \times 100^2$.

Remark 1: Although the proposed map cause higher storage consumption compared to certain existing approaches, e.g.,

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

OctoMap, it offers significantly faster update and sampling operations. In the ACCMPS, the local map represents only a limited range of the environment surrounding the UAV, in contrast to the extensive global map. With the subsequent proposed global map, the overall storage consumption are substantially reduced compared to other alternatives [7], [8], [9], [10].

B. Global Mapping

The global mapping sub-module is designed to construct an adaptive clustering weighted graph that retains only the essential information needed for the subsequent path planning module. This approach not only saves storage space but also enhances the speed of searching for the optimal initial path.

The global map is described as

$$\boldsymbol{G}^g = (\boldsymbol{V}, \boldsymbol{E}), \tag{3}$$

where

$$\begin{aligned} &\boldsymbol{V} \in \mathbb{R}^3, \\ &\boldsymbol{E} = \{(\boldsymbol{P}_1, \boldsymbol{P}_2, \boldsymbol{P}_m, l) | \boldsymbol{P}_1, \boldsymbol{P}_2, \boldsymbol{P}_m \in \mathbb{R}^3, l \in \mathbb{R}^+\}, \end{aligned} \tag{4}$$

and $\boldsymbol{V}$ is the set of cluster centers; $\boldsymbol{E}$ is the set of edges; $\boldsymbol{P}_1$ and $\boldsymbol{P}_2$ are two linkable cluster centers of the edge; $\boldsymbol{P}_m$ is the linkage point of the edge; l is $|\boldsymbol{P}_1\boldsymbol{P}_2|$.

We use adaptive clusters to determine $\boldsymbol{V}$,b and then adaptive clusters are described as

$$\begin{aligned} &\boldsymbol{C} = \{(\boldsymbol{V}, w, T_s, T, \boldsymbol{N}) | \boldsymbol{G} \in \mathbb{R}^3, w \in \mathbb{R}^+, T_s, T \in \mathbb{Z}^+\}, \\ &\boldsymbol{N} = \{(\boldsymbol{P}, \boldsymbol{C}) | \boldsymbol{P} \in \mathbb{R}^3\}, \end{aligned} \tag{5}$$

where $\boldsymbol{V}$ is the set of cluster centers; w is the weight of this cluster; T_s and T are success and total times of sampling based on this cluster; $\boldsymbol{N}$ is the set of nodes based on this cluster; and $\boldsymbol{P}$ is the position of the node.

The process of updating adaptive clusters is detailed in Algorithm 2, with the corresponding diagram presented in Fig. 4 (a). The sub-functions are described as follows.

- **InitClusterList**($\boldsymbol{C}$): Initilize all clusters in $\boldsymbol{C}$ that are in sight. Set T_s and T as 1, and remove all nodes in $\boldsymbol{N}$. Then create a new node in which $\boldsymbol{P}$ is $\boldsymbol{V}$ and add this node to $\boldsymbol{N}$.
- **SampleCluster**($\boldsymbol{C}$): This function is to sample a cluster in the cluster list C based on their weights. The weights of clusters are described as

$$\begin{aligned} &w = \hat{\theta} \cdot \hat{R}, \\ &\hat{\theta} = E(\theta | T_s \sim Be(T_s + 1, T - T_s + 1)) = \frac{T_s + 1}{T + 2}, \\ &\hat{R} = \frac{\pi r^3 - T_s \pi s^3}{\pi r^3}, \end{aligned} \tag{6}$$

where $\hat{\theta}$ is the estimated successful rate of sampling based on this cluster by Bayesian rule; $\hat{R}$ is the reciprocal of estimated occupied rate of this cluster; r is the ideal radius of each cluster; and s is the step size of sampling. This method prioritizes clusters with higher successful sampling rates by assigning them greater selection probabilities, thereby enhancing sampling efficiency. To prevent excessive dominance by a few clusters, the parameter $\hat{R}$ is introduced to maintain inter-cluster node balance. Notably, the aggregate weight of all clusters progressively decreases during sampling. The updating process terminates when this value falls below the threshold c_w, indicating complete coverage of the field-of-view (FOV) space.

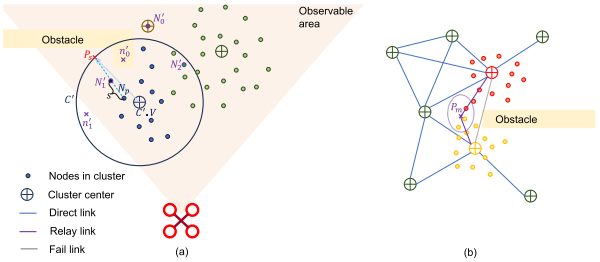


Fig. 4. Diagram of global mapping in 2D viewpoint. (a) Process of creating a node based on a cluster. (b) Process of building linkages of the global map.

- **SampleNewNode**($\boldsymbol{C}'$): This function is to sample a new node based on the cluster $\boldsymbol{C}'$ First, a point $\boldsymbol{P}_s$ of the surface of the ball with the center at $\boldsymbol{C}'.\boldsymbol{V}$ and radius r will be sampled uniformly. This can be described as

$$\begin{aligned} &\boldsymbol{P}_s = \boldsymbol{C}'.\boldsymbol{V} + r\boldsymbol{u}, \\ &s.t.\ |\boldsymbol{u}| = 1, \end{aligned} \tag{7}$$

After identifying the nearest node to $\boldsymbol{P}_s$, denoted as $\boldsymbol{N}^p$, we generate a new sample node $\boldsymbol{N}'$ by moving from $\boldsymbol{N}^p$ in the direction of $\boldsymbol{N}^p\boldsymbol{P}_s$ for a distance of s. Mathematically, this can be expressed as

$$\boldsymbol{N}'.\boldsymbol{P} = \boldsymbol{P}_s + \frac{s}{|\boldsymbol{N}^p.\boldsymbol{V}\ \boldsymbol{P}_s|}(\boldsymbol{P}_s - \boldsymbol{N}^p.\boldsymbol{V}). \tag{8}$$

- **CollisionTest**($\boldsymbol{N}', \boldsymbol{N}^p, \boldsymbol{G}^l$): Check whether the line segment between $\boldsymbol{N}'$ and $\boldsymbol{N}^p$ is blocked by obstacles marked in $\boldsymbol{G}^l$ by d_{obj}.
- $\boldsymbol{C}'$.**UpdateCluster**($b, \boldsymbol{N}'$): Change T and T_s in $\boldsymbol{C}$ correspondly based on whether this sample is success marked by b. If the sample is successful, update the center of nodes in this cluster $\boldsymbol{C}'.\boldsymbol{V}$.
- **FindNearestCluster**($\boldsymbol{N}'$): Find the nearest collision-free cluster center to $\boldsymbol{N}'$.
- **GetGrid**($\boldsymbol{C}^n.\boldsymbol{V}$): Obtain the index of $\boldsymbol{C}^n.\boldsymbol{V}$ in the local map $\boldsymbol{G}^l$.

Then, the process of updating adaptive clusters is introduced. At the beginning of a flight task, since no cluster exists in the global map, the cluster is initialized with a single node representing the UAV's position is created. Subsequent clusters are expanded based on this initial cluster. To improve sampling efficiency, a two-stage strategy is adopted: first, a cluster is sampled among all clusters according to their weight by (6); then, a node is sampled within the selected cluster.

To ensure that each cluster represents a relatively convex region, an adaptive strategy is proposed to determine whether the sampled free space should be explored and whether the node should be added to an existing cluster or used to create a new one. The following discussion refers to Fig. 4 and

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 UpdatingAdaptiveClusters($\boldsymbol{G}^l, \boldsymbol{G}^g, \boldsymbol{C}$)

```
InitClusterList(C);
while Sum(C.w) < c_w do
    C' ← SampleCluster(C);
    N', N^p ← SampleNewNode(C');
    if CollisionTest(N', N^p, G^l)= 1 then
        // Fail: n'_0 in Fig.3 (a) ;
        C'.UpdateCluster(0);
        continue;
    end
    C^n ← FindNearestCluster(N');
    if |C^n.V  N'| < c_c then
        if G^l_GetGrid(C^n.V).b_ob = 0 then
            // Fail: n'_1 in Fig.3 (a) ;
            C'.UpdateCluster(0);
        end
        else
            // Success: N'_0 in Fig.3 (a) ;
            new C^new;
            C^new.push_back(N');
            C^new.UpdateCluster(1, N');
            C.push_back(C^new);
        end
    end
    else
        if C^n is C' then
            // Success: N'_1 in Fig.3 (a) ;
            C'.push_back(N');
            C'.UpdateCluster(1, N');
        end
        else
            // Success: N'_2 in Fig.3 (a) ;
            C'.UpdateCluster(0);
            C^n.push_back(N');
            C^n.UpdateCluster(1, N');
        end
    end
end
```

Algorithm 2 and considers the following cases. (1) If the sampled node collides with obstacles (e.g., n'_0), connection between the sampled node and the nearest cluster center is blocked, or lies outside the UAV's field of view (e.g., n'_1), the node is rejected and the weight of the corresponding cluster is decreased. (2) If the sampled is too far away from the sampled cluster (e.g., N'_0), a new cluster is created containing only this node; its position is set as the cluster center, and its weight is initialized according to (6). (3) If the sampled node can be directly connected to the nearest cluster center (e.g., N'_1 and N'_2), it is added to that cluster, and the cluster weight is updated accordingly using (6).

To account for the impact of moving objects on the global map, at the beginning of each time step all cluster nodes are cleared using **InitClusterList**($\boldsymbol{C}$) in Algorithm 2, and sampling is re-expanded from the cluster centers obtained in the previous time step. This mechanism allows cluster centers to adaptively shift, enabling the global map to better reflect relatively convex regions in dynamic environments.

Remark 2: The proposed sampling rule effectively avoids blind sampling by reducing the probability of further sampling within a cluster as the number of failed attempts increases. This helps to prevent the algorithm from repeatedly sampling within enclosed or obstructed regions, thereby reducing redundant computations and enhancing overall efficiency. In addition, the probability of sampling decreases as the number of nodes in a cluster increases, effectively limiting the size of each cluster. Overall, the process of searching the moving space resembles sampling-based path planning algorithms, where moving spaces are represented by clusters. Therefore, compared to the PRM algorithm, our algorithm only searches the space accessible from the UAV, which significantly saves computational time.

It is worth emphasizing that, the proposed adaptive clustering strategy enables each cluster to represent a relatively convex region, as clusters are formed by expanding from their centers while ensuring collision-free node generation. In narrow environments, each space can be represented by one or some clusters. As points that can represent isolated spaces, centers of clusters are also maximally distanced from surrounding obstacles in narrow spaces, making them candidate path nodes in the graph.

After updating the cluster list, we construct the adaptive clustering weighted map by examining the linkages between clusters. The update process of the global map is detailed in Algorithm 3, while the diagram of the constructed global map is illustrated in Fig. 4 (b). The sub-functions are described as

- **FindMidPoint**($\boldsymbol{C}_i.\boldsymbol{V}, \boldsymbol{C}_j.\boldsymbol{V}$): The function returns the center of all nodes in $\boldsymbol{C}_i$ or $\boldsymbol{C}_j$, which can link to both $\boldsymbol{C}_i.\boldsymbol{V}$ and $\boldsymbol{C}_j.\boldsymbol{V}$ with no collision.

Remark 3: In updating the global map, we preserve all linkages between clusters out of the local map to save unnecessary computational resources and re-check those in the local map to deal with moving objects. The global map only stores useful information for UAV navigation, i.e., linkages of convex sub-spaces, saving much storage resources compared to other maps, e.g., [7], [8], [9], and [10]. Furthermore, the subsequent path planning module can identify the initial path much more quickly in the graph-based map than through blind searches in a grid-based map. This proposed map effectively bridges the gap between the mapping and planning modules. In addition, compared with traditional maps, the proposed map has more advantages in terms of saving computing resources and storage space, as the optimal initial path can be obtained more directly through the graph structure. This content will be discussed in detail in the next section.

C. Storage Consumption

In the proposed ACCMPS, we use a 3D grid map only to represent the nearby environments, and the proposed graph-based map for the global environment. The storage consumption of the proposed mapping strategy is much lower than traditional grid maps, and the advantage will be amplified with the expansion of the environment. Then, a detailed discussion of storage consumption is provided based on Fig. 5.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 UpdatingLinkages(G^g, G^l)

```
for E_k in E do
    if G^l_{GetGrid(E_k.P_1)}.b_ob = 1 or G^l_{GetGrid(E_k.P_2)}.b_ob = 1 then
        erase E_k;
    end
end
for i from 0 to size(C) do
    for jfromi to size(C) do
        if size(C_i.N) = 0 or |C_i.V C_j.V| < c_d then
            // Fail link in Fig.3 (b);
            continue;
        end
        if CollisionTest(C_i.V, C_j.V, G^l) = 0 then
            // Success link: direct link in Fig.3 (b);
            E.push_back(C_i.V, C_j.V, _, |C_i.V C_j.V|);
        end
        else
            P_m = FindMidPoint(C_i.V, C_j.V);
            if P_m exist then
                // Success link: relay link in Fig.3 (b);
                E.push_back(C_i.V, C_j.V, P_m, |C_i.V C_j.V|);
            end
        end
    end
end
```

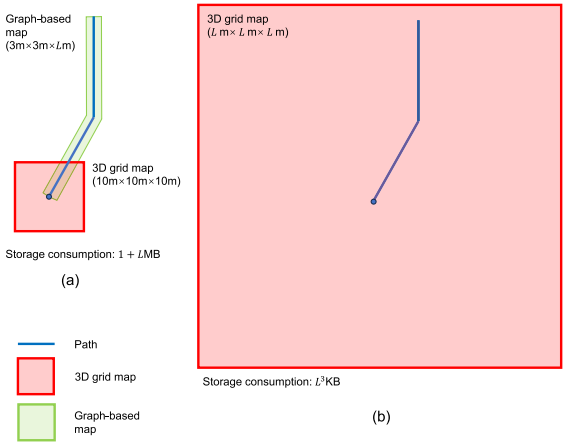


Fig. 5. Diagram of map storage. (a) The map storage used in ACCMPS uses a 3D grid map to represent the nearby environment and a proposed graph-based map for the global environment. (b) The storage consumption of using a 3D grid map solely to represent the environment is much larger.

For the 3D grid map, a certain amount of storage resources will be allocated in advance to store the map. With the assumption that the grid size is 0.1m and using char (1B) to represent the probability that one grid is occupied by obstacles, the storage consumption is N KB, where N is the volume (m^3) of the preset map.

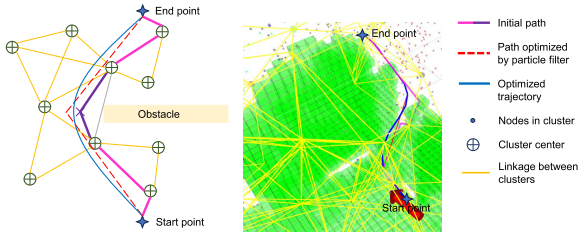


Fig. 6. Diagram of planning in 2D and 3D viewpoints.

Concerning the proposed graph-based map, we only need to store the centers of clusters that can represent a relatively convex space and the linkages among different centers. For each center, we only need to store its position and the index of the linked clusters with less than 10B. Based on experiments, each cubic meter of space can be represented by 10 clusters with about 100B. Additionally, since the graph-based map is generated by the process of the UAV moving, not preset, with the assumption that the sight width is 3m, the total storage is about L MB, where L is the total length of the path that the UAV's battery can support to fly.

Therefore, as shown in Fig. 5(a), when employing a $10m \times 10m \times 10m$ 3D grid map to represent the local environment and the proposed graph-based map for the global environment, the total storage requirement of the ACCMPS mapping system is approximately $1 + L$ MB.

Remark 4: The $1+L$ MB storage consumption is acceptable for the majority of mobile devices, even when the planned path is long. If we use the 3D grid map solely, as can be seen in Fig. 5 (b), in unknown environments, we cannot know how large the map is with the only input of the initial and target points. Therefore, to maximize the exploration ability, i.e., the battery can support the UAV to fly L m, the preset map should be L^3 KB, which is unbearable for most devices. This issue will be further discussed in Experiments section.

IV. Planning

The planning module uses the local and global maps built in the mapping module to generate a high-quality trajectory for the UAV to follow. The initial path searching sub-module is first employed to find the shortest path in the adaptive clustering weighted graph. Then, a particle filter is used to optimize the path by iteration. Finally, a trajectory optimization problem is formulated and solved using a gradient-based optimizer to generate a high-quality B-spline trajectory. The process of planning is shown in Fig. 6.

A. Initial Path Searching in Graph

The first step of the planning module is to find the initial path in the built adaptive clustering weighted graph. The start and end clusters are described as

$$\boldsymbol{V}_s = \min_{\boldsymbol{V}' \in \boldsymbol{C}.\boldsymbol{V}} |\boldsymbol{P}_t \boldsymbol{V}'| + 2|\boldsymbol{V}' \boldsymbol{T}_{wd}|, \tag{9}$$

$$\boldsymbol{V}_e = \min_{\boldsymbol{V}' \in \boldsymbol{C}.\boldsymbol{V}} |\boldsymbol{P}_t \boldsymbol{V}'| + |\boldsymbol{V}' \boldsymbol{T}_{wd}|,$$

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

Algorithm 4 ParticleFilter($\boldsymbol{P}, r, k, M, N, \boldsymbol{G}^l$)

Notation: r is the initial search radius; k is its decrease rate, M is the round of iterations, N is the number of sample points in each iteration;
$\boldsymbol{P} \leftarrow$ **LOSChecking**($\boldsymbol{P}$);
for m *from 1 to* M **do**
  for $\boldsymbol{P}_i$ *in* $\boldsymbol{P}$ **do**
    if *CollisionTest*($\boldsymbol{P}_{i-1}, \boldsymbol{P}_{i+1}, \boldsymbol{G}^l$) $= 0$ **then**
      erase $\boldsymbol{P}_i$;
      continue;
    end
    $\boldsymbol{P}^c \leftarrow$ **SamplePoints**($\boldsymbol{P}_i, r, N$);
    $\boldsymbol{P}_i \leftarrow$ **FindBestSample**($\boldsymbol{P}_{i-1}, \boldsymbol{P}_{i+1}, \boldsymbol{P}^c$);
  end
  $r \leftarrow r \cdot k$;
end
$\boldsymbol{P} \leftarrow$ **PathSubdivision**($\boldsymbol{P}, s$);

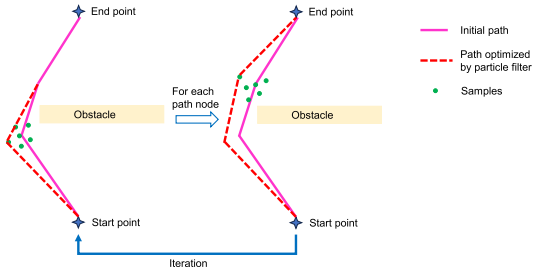


Fig. 7. Diagram of one round of particle resampling and refinement. This figure shows an intermediate particle filter result, where path quality is further refined in a subsequent optimization stage; final smooth and executable trajectories are presented in the experiments.

$$s.t.\ \boldsymbol{G}^l_{\mathbf{GetGrid}(\boldsymbol{V}')}.b_{ob} = 0 \ \vee\ |\boldsymbol{V}'\boldsymbol{P}_{tar}| < c_c. \tag{10}$$

The initial path is the minimal route from $\boldsymbol{V}_s$ to $\boldsymbol{V}_e$, generated by the Dijkstra's algorithm, denoted as

$$\begin{aligned} &\boldsymbol{P} = \{\boldsymbol{T}_{wd}, \boldsymbol{V}_s, \boldsymbol{P}_3, \cdots, \boldsymbol{P}_{m-2}, \boldsymbol{V}_e, \boldsymbol{P}_t\}, \\ &\boldsymbol{P}_i \in \mathbb{R}^3. \end{aligned} \tag{11}$$

Remark 5: As previously mentioned, the global mapping module represents the convex sub-spaces and their linkages using a weighted graph. In the planning module, the traditional Dijkstra's algorithm [36] can be employed to efficiently find the shortest path. In contrast to traditional algorithms that perform blind sampling in unknown spaces represented by grid maps, the proposed method facilitates more effective pathfinding within known environments.

B. Generating Path by Particle Filter

The second step of the planning module is to optimize the initial path using the particle filter. The process of the particle filter optimization is outlined in Algorithm 4, the diagram is shown in Fig. 7, and the sub-functions are described as

- **LOSChecking**($\boldsymbol{P}$): Apply a line-of-sight (LOS) checking strategy to remove redundant path points in $\boldsymbol{P}$ by eliminating intermediate nodes whose direct connections to non-adjacent nodes are collision-free.
- **SamplePoints**($\boldsymbol{P}_i, r, N$): Sample N points in a ball based on $\boldsymbol{P}_i$ with the radius r uniformly.
- **FindBestSample**($\boldsymbol{P}_p, \boldsymbol{P}_n, \boldsymbol{P}^c$): This function is to find the best point $\boldsymbol{P}_o$ among all sample points, which is described as

$$\begin{aligned} \boldsymbol{P}_o &= \min_{\boldsymbol{P}' \in \boldsymbol{P}^c} w_1 L_l + w_2 L_z + w_3 L_d, \\ L_l &= |\boldsymbol{P}_p\boldsymbol{P}'| + |\boldsymbol{P}'\boldsymbol{P}_n|, \\ L_z &= |(\boldsymbol{P}_p - \boldsymbol{P}')\begin{bmatrix}0\\0\\1\end{bmatrix}| + |(\boldsymbol{P}' - \boldsymbol{P}_n)\begin{bmatrix}0\\0\\1\end{bmatrix}|, \\ L_d &= \frac{1}{d_{min_p}} + \frac{1}{d_{min_n}}, \\ &s.t.\ \min(d_{min_p}, d_{min_n}) > c_d, \end{aligned} \tag{12}$$

  where w_1, w_2, and w_3 are weights; d_{min_p} and d_{min_n} are $\min_{\boldsymbol{P}'' \in \boldsymbol{P}_n\boldsymbol{P}'} \boldsymbol{G}^l_{\mathbf{GetGrid}(\boldsymbol{P}'')}.b_{ob}$ and $\min_{\boldsymbol{P}'' \in \boldsymbol{P}'\boldsymbol{P}_p} \boldsymbol{G}^l_{\mathbf{GetGrid}(\boldsymbol{P}'')}.b_{ob}$, respectively.
  It is interesting to note that, in addition to the total length of the path and the minimum distance to obstacles, we add the additional penalty to changes in height. This is because ascending and descending consumes more energy than navigating around obstacles horizontally.
- **PathSubdivision**($\boldsymbol{P}, s$): Subdivide the path $\boldsymbol{P}$ by inserting additional points such that the length of each path segment is approximately to s.

Before conducting gradient-based optimization, the path is roughly optimized using the particle filter, primarily for the following three reasons. First, the initial path identified in the graph may be significantly distant from the final trajectory, and the overall space is non-convex, which hinders the efficiency of gradient-based optimization. In contrast, the particle filter excels at handling non-convex problems. Second, within the particle filter framework, nodes can be easily added or removed, allowing the algorithm to eliminate unnecessary path points more effectively than gradient-based methods. Third, the path should consider the distance from obstacles, and gradients may not be defined in the local map represented by the 3D grid map. The absence of a gradient can lead to failures in gradient-based algorithms, making the particle filter a more suitable choice in this context.

However, the motion space of UAVs is inherently high-dimensional, involving position, pose, and velocity, which makes direct application of sampling-based methods such as particle filters computationally inefficient. In addition, to achieve high path quality and smoothness, the final trajectory often consists of densely distributed path points, which further increases the required number of samples and leads to unacceptable computational and memory costs.

In ACCMPS, these challenges are effectively alleviated by limiting the role of the particle filter. Specifically, the particle filter is only used to refine the initial path generated by the path search module, with the goal of improving its suitability for subsequent optimization. Sampling is performed solely in the 3D positional space, while the costs associated with pose and velocity variations are handled in the following optimization module. This design significantly reduces the dimensionality

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

of the sampling space and addresses the first challenge. To mitigate the second issue, a line-of-sight (LOS) checking strategy is employed to remove redundant path points, thereby reducing the number of required samples. In practice, the particle filter accounts for less than 10% of the total computational time of ACCMPS.

By generating a near-optimal initial path, the proposed method effectively reduces the optimization gap and mitigates the adverse effects of non-convexity in the decision space. As a result, the subsequent gradient-based path optimization can converge more efficiently toward a high-quality solution.

Remark 6: It is also worth noting that although the weights w_1, w_2, and w_3 are determined through empirical tuning, the proposed system exhibits strong robustness to parameter selection. This robustness is demonstrated in the experimental section, where the same set of weights is consistently used across all experiments conducted in different environments. Moreover, the refined path generated by the particle filter is directly passed to the subsequent optimization module to produce the final smooth and continuous trajectory, as introduced in the following subsection.

C. Generating Trajectory by Gradient-Based Optimization

The gradient-based optimization is employed to generate a high-quality trajectory for the UAV to follow. We employ a cubic B-spline, which is composed of cubic polynomial segments, i.e., a B-spline of order 4 in precise mathematical terms, to ensure C2 continuity for smooth and dynamically feasible trajectories. Before this optimization, we establish the initial trajectory based on the path optimized by the particle filter, which is described as

$$\boldsymbol{B}(t) = \sum_{i=1}^{N_c} \boldsymbol{N}_{i,3}(t)\boldsymbol{Q}_i, \tag{13}$$

where

$$\begin{aligned} &\boldsymbol{Q} = \{\boldsymbol{Q}_1, \boldsymbol{Q}_2, \cdots, \boldsymbol{Q}_{N_c}\}, \\ &U = \{0,0,0,0,u_1,u_2,\ldots,u_{N-3},u_{N-3},u_{N-3},u_{N-3}\}, \end{aligned} \tag{14}$$

and $\boldsymbol{N}$ is the cubic B-spline basis function [21]. $\boldsymbol{Q}$ is generated from $\boldsymbol{P}$ with a fixed distance between two control points, and U is generated based on $\boldsymbol{Q}$ with the same time interval Δt to control the velocity of the UAV.

The cost function L contains L_v about the change of velocity and L_g about the trajectory's deviation of $\boldsymbol{P}$, which is described as

$$\begin{aligned} L &= \min_{\boldsymbol{P}} w_4 L_v + w_5 L_g, \\ L_v &= \sum_{i=2}^{m-1} \left| \frac{\boldsymbol{Q}_{i+1} - \boldsymbol{Q}_i}{\Delta t} - \frac{\boldsymbol{Q}_i - \boldsymbol{Q}_{i-1}}{\Delta t} \right|, \\ L_g &= \sum_{i=1}^{m} \min_j \mathrm{Dist}(\boldsymbol{Q}_i, \boldsymbol{P}_j\boldsymbol{P}_{j+1}), \end{aligned} \tag{15}$$

where $\mathrm{Dist}(\boldsymbol{Q}_i, \boldsymbol{P}_j\boldsymbol{P}_{j+1})$ is the distance from point $\boldsymbol{Q}_i$ to the line segment $\boldsymbol{P}_j\boldsymbol{P}_{j+1}$. The classical Dogleg algorithm [37] is used to optimize this problem.

Next, we generate knots U to fit the requirement of kinematic constraints considering the given maximum velocity v_{max}, acceleration a_{max}, and angular velocity ω_{max}. Based on the definition in (13) the velocity, acceleration, and angular velocity can be respectively described as

$$\begin{aligned} &\mathbf{C}'(u) = \sum_{i=0}^{n} \boldsymbol{N}'_{i,p}(u)\boldsymbol{Q}_i, \quad \mathbf{C}''(u) = \sum_{i=0}^{n} \boldsymbol{N}''_{i,p}(u)\boldsymbol{Q}_i, \\ &\omega(u) = \frac{\|\mathbf{C}'(u) \times \mathbf{C}''(u)\|}{\|\mathbf{C}'(u)\|^2}. \end{aligned} \tag{16}$$

For satisfying $\|\mathbf{C}'(u)\| \le v_{\max}$, $\|\mathbf{C}''(u)\| \le a_{\max}$, and $\omega(u) \le \omega_{max}$, we have

$$\begin{aligned} &\Delta u \ge \frac{\|\Delta \boldsymbol{Q}_i\|}{v_{\max}}, \quad \Delta u \ge \sqrt{\frac{\|\Delta^2 \boldsymbol{Q}_i\|}{a_{\max}}}, \\ &\Delta u \ge \frac{\|\mathbf{C}'(u) \times \mathbf{C}''(u)\|}{\omega_{\max}\|\mathbf{C}'(u)\|^2}, \end{aligned} \tag{17}$$

where $\Delta^2 \boldsymbol{Q}_i = \boldsymbol{Q}_{i+2} - 2\boldsymbol{Q}_{i+1} + \boldsymbol{Q}_i$.

Therefore, an iteration is leveraged from the initial knot vector $\boldsymbol{U}$, and for each knot span $[u_i, u_{i+1}]$, compute the required δu_{min} based on the following formula:

$$\Delta u_{\min} = \max\left(\frac{\|\Delta \boldsymbol{Q}_i\|}{v_{\max}}, \sqrt{\frac{\|\Delta^2 \boldsymbol{Q}_i\|}{a_{\max}}}, \frac{\|\mathbf{C}' \times \mathbf{C}''\|}{\omega_{\max}\|\mathbf{C}'\|^2} \right). \tag{18}$$

If $\Delta u < \Delta u_{min}$, adjust $u_{i+1} = u_i + \Delta u_{min}$. According to the above processes, the B-spline can fit the requirement of kinematic constraints after adjusting U in the iteration.

Remark 7: Since the path optimized by the particle filter is close to the optimal path and the optimization space is nearly convex, a gradient-based optimizer is utilized to refine the final trajectory. While the particle filter optimizes the distance to obstacles and the total length of the trajectory, the gradient-based optimization primarily focuses on refining the change in velocity, thereby producing a smoother trajectory. This approach also helps avoid issues related to the disappearance of the gradient with respect to obstacle distances.

V. Experiments

In this section, ACCMPS is implemented on a UAV platform and evaluated in various environments. First, tests in a narrow environment assess its path planning performance in terms of time consumption and optimal path probability against other algorithms. Second, experiments in a large indoor environment focus on storage usage and replanning time. Finally, a large outdoor environment with diverse scenarios demonstrates ACCMPS's real-world applicability.

A. Platform Configuration and Parameter Settings

We design a UAV to evaluate the performance of ACCMPS and compare it against other mapping and planning algorithms. As shown in Fig. 8, the hardware configuration of the UAV contains an Intel RealSense D455 depth camera to perceive the environment, and the Pixhawk 4 flight controller with PX4 to control the motor of the UAV. The onboard computer is Lattepanda Sigma with Ubuntu 24.04 LTS as its operating

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

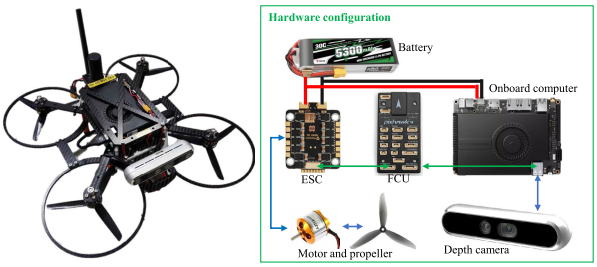


Fig. 8. The hardware configuration of the UAV.

TABLE I
KEY PARAMETERS OF THE USED UAV AND SENSORS

Parameter	Value
UAV Distance of axles	$0.33m$
UAV Weight	$1.8kg$
UAV maximum velocity	$2.0m/s$
UAV maximum accelerate	$0.5m/s^2$
UAV maximum angular velocity	$45°/s$
Depth camera sight distance	$4.0m$
Depth camera field of vision	$87° \times 58°$

TABLE II
KEY PARAMETERS OF ACCMPS

Character	Parameter	Value
-	Local map: grid size	$0.1m \times 0.1m \times 0.1m$
-	Local map: sub-map size	$1m \times 1m \times 1m$
-	Local map: whole map size	$10m \times 10m \times 10m$
c_c	Global map: maximum cluster radius	$1.0m$
c_d	Global map: maximum edge length	$3.0m$
s	Global map: step size of sampling	$0.3m$
r	Planning: PF initial search radius	$1.0m$
M	Planning: PF rounds of iterations	10
w_1, w_2, w_3	Planning: PF weights	$1.0, 10.0, 3.0$
w_4, w_5	Planning: optimization weights	$1.0, 0.1$
-	Global map: optimization iterations	10
-	minimum safe distance to obstacles	$0.3m$

system, and the robotic system is implemented using ROS2 Jazzy distribution. The key parameters settings of the UAV and onboard sensors used in the following experiments are summarized in Table I, while the main parameters for the proposed ACCMPS are listed in Table II.

To make a fair comparison, the same parameters are shared between the ACCMPS and the compared algorithms.

B. Narrow Environmental Experiments

First, we test the performance of ACCMPS in a narrow environment, where two walls separate from each other, leaving a narrow access. This scenario is designed to intuitively evaluate

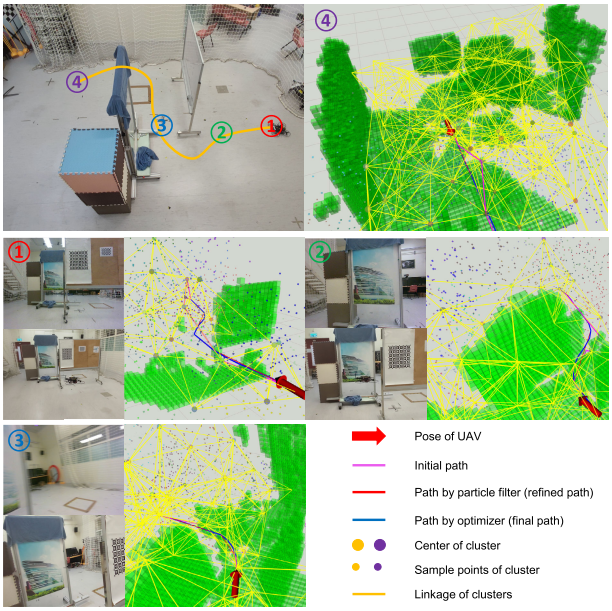


Fig. 9. Results of ACCMPS's evaluation in a real-world environment.

the quality and time consumption of the proposed method in local planning compared to other methods. As illustrated in Fig. 9, the particle-filter path represents an intermediate refined path used as the input to the optimization module, while the optimized path corresponds to the final smooth and executable trajectory. Despite the limited space, ACCMPS can still generate a smooth trajectory through the narrow passage while maintaining a safe distance from surrounding obstacles.

1) Comparison With Other Algorithms: Then, we compare the result of local planning between different algorithms, i.e., RRT*-smart [13], Improved Bi-PRM [38], A* [15], Theta* [16], and jump point search (JPS) [17], at Scenario 2 in the environment, which is shown in Fig. 10. As a result, our algorithm makes sure to find the optimal path through the access with stable and reasonable time consumption. As for the RRT*-smart algorithm, although the time consumption is low, it seldom finds the optimal path. For Bi-PRM, using too few sample points often results in failure to find the optimal path, while increasing the number of points leads to unacceptable time consumption. As for A* algorithm, its computation time is approximately 10 times longer than that of our approach. Next, we block the passage to force the path planner to circumvent a large wall. In this scenario, the A* algorithm requires significant time to explore the area beyond the wall, and a similar issue is observed with the APF algorithm. In contrast, our algorithm is able to avoid the large wall efficiently without incurring additional computational overhead. As updated variants of A*, both Theta* and JPS are not particularly well suited for complex 3D environments. Specifically, Theta* can generate paths faster than standard A*, but it does not guarantee optimality, which may lead to suboptimal trajectories in cluttered 3D spaces. On the other hand, JPS ensures path optimality, but its performance is significantly constrained by the complexity of 3D environments, resulting in computation times more than five times higher than our proposed ACCMPS.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

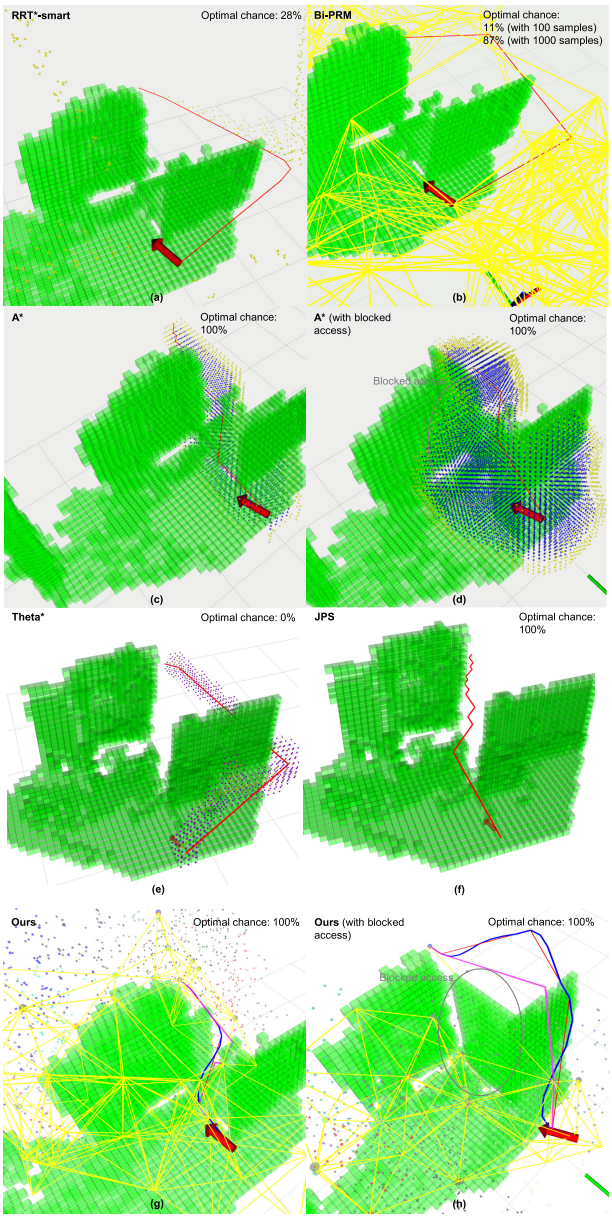


Fig. 10. Comparison between different algorithms in a narrow environment. The legend is same as Fig. 9. (a) RRT*-smart can generate an im-optimal path in a short time; (b) Bi-PRM can generate an im-optimal path in a short time by fewer samples; however, increasing the probability of generating the optimal needs to add additional samples, making time consumption unacceptable; (c) A* can find the optimal in a reasonable time;(d) More grids will be calculated when encountering large obstacles, leading time consumption being unacceptable; (e) Theta* can find the unoptimal path in a short time; (f) JPS can find the optimal path but is not suitable for complex 3D environment; (g) Our algorithm can find the optimal path much faster than A*; (h) The time consumption of our algorithm will nearly not be influenced by obstacle.

Based on the above analysis, our algorithm integrates the merit of both RRT*-smart and Bi-PRM algorithms and conquers their drawbacks. Compared to Bi-PRM's unoriented sampling, the adaptive cluster-based sampling of our algorithms can represent each convex space more effectively, enabling the algorithm to find the optimal path with fewer nodes. In comparison to RRT*-smart, by updating the parameters of adaptive clusters based on sampling results, our algorithm significantly improves sample efficiency by avoiding

TABLE III

AVERAGE TIME COST OF EACH MODULE OF ALGORITHMS IN A NARROW ENVIRONMENT (↑: OTHER CASES; ↓: WORST 10% CASES)

Algorithm		Local mapping	Global mapping	Path planning	Sum	Optimal
RRT*-smart	↑	0.011s	-	0.013s	0.024s	28%
	↓	0.012s	-	0.018s	0.030s	
Bi-PRM (100 samples)	↑	0.011s	-	0.010s	0.021s	11%
	↓	0.012s	-	0.011s	0.023s	
Bi-PRM (1000 samples)	↑	0.011s	-	5.1s	5.111s	87%
	↓	0.012s	-	5.3s	5.312s	
A*	↑	0.011s	-	0.30s	0.311s	100%
	↓	0.012s	-	0.31s	0.322s	
A* (block access)	↑	0.011s	-	3.4s	3.411s	100%
	↓	0.012s	-	3.5s	3.512s	
Theta*	↑	0.011s	-	0.021s	0.032s	0%
	↓	0.012s	-	0.023s	0.034s	
JPS	↑	0.011s	-	0.19s	0.201s	100%
	↓	0.012s	-	0.21s	0.212s	
Ours	↑	0.011s	0.013s	0.008s	0.032s	100%
	↓	0.012s	0.015s	0.009s	0.036s	
Ours (block access)	↑	0.011s	0.014s	0.008s	0.033s	100%
	↓	0.012s	0.015s	0.010s	0.037s	

blind sampling in trap regions. The improved sampling efficiency enables our algorithm to explore the entire perceived space in real time, making generating an optimal path more promising. In addition, since UAV navigation requires real-time path planning, the stability of time consumption is more critical than the average execution time. As can be seen from Table III, the time consumption of our algorithm is in the range of 0.033s to 0.037s in all cases, much lower than the worst cases of other algorithms.

2) Relative Convexity and Convex Rate: To formalize the notion of relatively convex sub-spaces used in this work, we introduce a quantitative metric termed *convex rate*, which evaluates the degree to which a cluster approximates a convex free-space region in the presence of obstacles. Rather than enforcing strict geometric convexity, the proposed metric captures a relative and practical notion of convexity that is directly relevant to sampling-based path planning.

Each cluster C is associated with a geometric center c. The convex rate of a cluster is defined as the proportion of nodes within the cluster that can be directly connected to the cluster center via a collision-free straight-line path. Formally, the convex rate is defined as

$$\mathrm{CR}(C) = \frac{1}{|C|} \sum_{v \in C} \mathbb{I}(\mathrm{LOS}(v, c)), \tag{19}$$

where $\mathrm{LOS}(v, c)$ denotes the existence of a line-of-sight; collision-free path between node v and the cluster center c; $\mathbb{I}(\cdot)$ is the indicator function that equals 1 if the condition is satisfied and 0 otherwise. A higher convex rate indicates that most nodes in the cluster are mutually visible from the center,

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

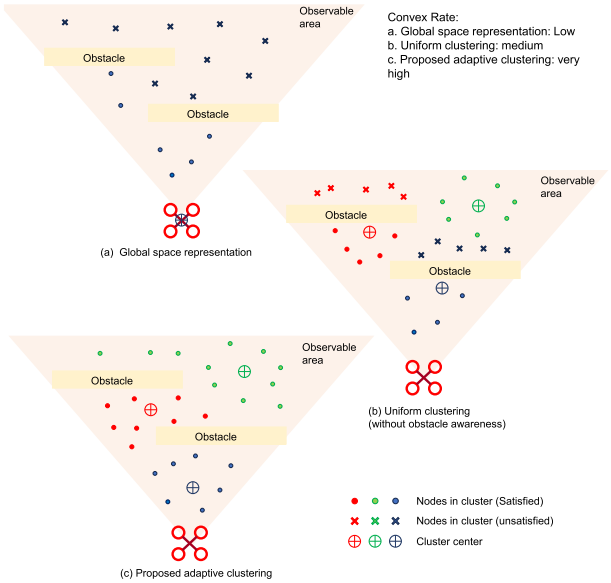


Fig. 11. Conceptual 2D schematic illustrating the convex rate under different space representations. The illustration is provided for visualization purposes only, while all clustering and evaluations are performed in the full 3D environment.

implying that the cluster closely resembles a star-shaped region and can therefore be regarded as a relatively convex sub-space.

As illustrated in Fig. 11, the convex rate quantifies the visibility connectivity between cluster nodes and the cluster center. The figure provides a conceptual 2D schematic to illustrate the geometric intuition of the metric, and does not represent experimental data. All convex rates reported below are computed on full 3D maps.

Based on the quantitative evaluation in Scenario 2 (Fig. 9), when the entire free space perceived by the depth camera is represented as a single region centered at the UAV, the average convex rate is 30.2%, which leads to inefficient exploration and blind sampling in non-convex free space for planners such as RRT. When an even clustering and sampling strategy is applied without explicitly considering obstacle-induced separations, the average convex rate increases to 78.7%, comparable to PRM-style planning. By contrast, the proposed adaptive clustering strategy achieves a convex rate of up to 99.2%, indicating that the resulting clusters exhibit significantly stronger relative convexity. This property explains why the proposed method is able to consistently identify near-optimal paths with substantially reduced planning time.

C. Large-Scale Indoor Environmental Experiments

Next, ACCMPS is tested in a long corridor shown in Fig. 12 to evaluate its performance in terms of map storage and replanning based on the built graph map. The corridor is about 60m long, containing two 90° turns near the start point and one 180° turn near the target point.

The scenario is very challenging for computation time as the long global path and some aggressive turnings require more exploration space and increase the failure exploration rate, respectively. In this scenario, ACCMPS represents the



Fig. 12. Evaluation of ACCMPS in a large indoor environment. The legend is same as Fig. 9.

surrounding and global environments by the 3D grid map and the proposed graph-based map effectively. The planning module generates high-quality paths in real-time by the map, leading the UAV to the target point safely.

As illustrated in Fig. 13, we compare the proposed ACCMPS with EGO-Planner [18] under three representative navigation scenarios. In the first scenario, a narrow corridor is located in front of the UAV. ACCMPS generates a smooth and nearly straight trajectory after ascending to approximately 1m, while maintaining sufficient clearance from surrounding obstacles. In contrast, the trajectory produced by EGO-Planner exhibits an S-shaped pattern. This behavior can be attributed to its trajectory optimization formulation, in which each path point is mainly influenced by its nearest obstacle. In constrained environments, such as narrow corridors, this may introduce a lateral bias, as distances to other nearby obstacles are not jointly considered during optimization. By contrast, ACCMPS efficiently maintains a distance-aware map, where each grid cell explicitly stores the distance to its nearest obstacle, enabled by the proposed mapping data structure and

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

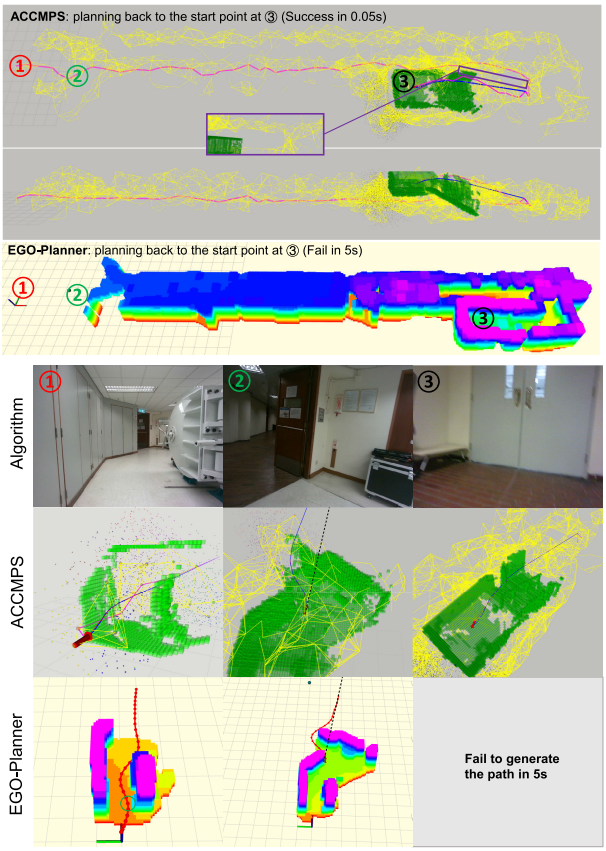


Fig. 13. Comparison between ACCMPS and EGO-Planner in a large indoor environment. The legend is same as Fig. 9.

update strategy, allowing more consistent obstacle-aware planning. In the second scenario, both planners successfully avoid an obstacle directly ahead of the UAV. However, after obstacle avoidance, the trajectory generated by EGO-Planner returns to the gray reference line shown in Fig. 13. This is because EGO-Planner sets the intermediate target of the local planner at a fixed distance (5.5m) along the line segment connecting the UAV and the global goal to reduce computational cost, which prioritizes local efficiency over global optimality. In contrast, ACCMPS maintains a global planning perspective and proceeds directly toward the global target after bypassing the obstacle, thereby eliminating the redundant detour. In the third scenario, after the UAV reaches the vicinity of the global target, both planners are required to replan a path back to the initial position. Leveraging the previously constructed graph-based map, ACCMPS completes the replanning within 0.05s. It is notable that even without storing the storage-consuming global grid-based map, ACCMPS can also find the global long-range path by the proposed global graph map and optimize the local path by the local grid-based map. Under this dead-end configuration, EGO-Planner fails to generate a feasible path within 5s. This configuration is particularly challenging for planners relying on grid-based global search, as the narrow corridor significantly limits feasible expansion directions. EGO-Planner employs a revised A*-based global planner operating on an occupancy grid, which struggles to efficiently resolve such scenarios. We further evaluated

TABLE IV

STORAGE CONSUMPTION OF EACH MAP IN A LARGE INDOOR ENVIRONMENT (V: GRID MAP; G: GRAPH MAP)

Map	Corresponding Algorithms		Storage	Total
3D grid map (300m×300m×300m)	RRT*-smart Bi-PRM A* EGO-Planner	V	27GB	27GB
		G	0	
3D grid map (100m×100m×10m)	RRT*-smart Bi-PRM A* EGO-Planner	V	100MB	100MB
		G	0	
3D octomap	RRT*-smart Bi-PRM A*	V	48.3MB	48.3MB
		G	0	
Ours	ACCMPS	V	1MB	16.7MB
		G	15.7MB	

RRT*-Smart and Bi-PRM under the same conditions, and neither method was able to produce a valid path within 5s.

Regarding the mapping storage consumption reported in Table IV, as analyzed in the Mapping section, the memory footprint of ACCMPS consists of a 1 MB 3D grid map and a 15.7 MB graph-based map. In contrast, planners that rely solely on a 3D occupancy grid map, such as EGO-Planner, maintain a large-scale probabilistic occupancy representation of the environment, in which each grid cell stores its occupancy state. In unknown environments, such a grid-based representation typically requires allocating a sufficiently large map in advance to ensure adequate exploration capability within the UAV's operational limits. For instance, assuming an average flight speed of 1 m/s and a flight duration of 5 minutes, the UAV may traverse a total path length of approximately 300 m. To cover the corresponding workspace using a uniform grid with a resolution of $0.1\text{m} \times 0.1\text{m} \times 0.1\text{m}$, a $300\text{m} \times 300\text{m} \times 300\text{m}$ grid map would be required, resulting in a storage consumption of approximately 27 GB, which is impractical for most onboard computing platforms. Even when partial prior knowledge of the environment is available and the map size is reduced to $100\text{m} \times 100\text{m} \times 10\text{m}$, the storage requirement remains around 100 MB, which still exceeds the memory footprint of ACCMPS. Alternatively, a 3D OctoMap representation could be employed to reduce memory usage. To fully cover a $300\text{m} \times 300\text{m} \times 300\text{m}$ space at a resolution of 0.1m, the resulting octree would require a depth of 12, leading to a storage consumption of approximately 48.3 MB. Although this is significantly smaller than that of a dense grid map, it remains about three times larger than the storage required by ACCMPS. Moreover, the increased tree depth introduces additional computational overhead for occupancy queries and map updates, which can adversely affect real-time performance.

As for the proposed ACCMPS, we only maintain a local $10\text{m} \times 10\text{m} \times 10\text{m}$ 3D grid map, which consumes only 1 MB, together with a compact graph-based map to represent the global environment. The graph-based map stores only the centers of clusters and the linkages among them. In the large-scale experimental scenario reported in Table IV, the graph contains fewer than 1500 cluster centers and their

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

TABLE V
STORAGE CONSUMPTION OF EACH MAP IN A LARGE OUTDOOR ENVIRONMENT (V: GRID MAP; G: GRAPH MAP)

Map	Corresponding Algorithms		Storage	Total
3D grid map (300m×300m×300m)	RRT*-smart Bi-PRM A* EGO-Planner	V	27GB	27GB
		G	0	
3D grid map (100m×100m×10m)	RRT*-smart Bi-PRM A* EGO-Planner	V	100MB	100MB
		G	0	
3D octomap	RRT*-smart Bi-PRM A*	V	36.7MB	36.7MB
		G	0	
Ours	ACCMPS	V	1MB	19.6MB
		G	18.6MB	

corresponding edges, resulting in a memory footprint of 15.7 MB. It is worth noting that this graph size is obtained in a highly cluttered environment with a flight distance of approximately 60 m. Assuming a constant flight speed of 1 m/s, even for a long-duration mission of 30 minutes (corresponding to a total flight distance of about 1800 m), the resulting graph-based map is expected to remain well below 500 MB. Therefore, the proposed representation can scale to long-range and long-duration exploration missions without incurring prohibitive memory consumption.

D. Large-Scale Outdoor Environmental Experiments

Additionally, we evaluate ACCMPS's performance in a large-scale outdoor environment, where the initial point is behind a column and the target point is in a grove. The environment contains several typical scenarios, such as buildings, grassland, roads, and grove, which can evaluate the performance of the UAV navigation system in the real world comprehensively. From Fig. 14, the proposed ACCMPS enables the UAV to avoid the column in scenario 1, crossing the road in scenarios 2 and 3, avoiding the shrub in scenario 4, and crossing the narrow access between two trees with no collision in scenarios 5 and 6.

Similarly, we compare the time consumption of finding the path from the target point to the initial point at the end of the navigation. In this open space, Bi-PRM and A* cannot find the path in 10s and RRT*-smart requires 1.21s to find a path, whereas the proposed ACCMPS algorithm achieves the same task in just 0.04s. In terms of storage consumption shown in Table V, despite the fixed consumption of the 3D grid map, the 3D octomap is two times larger than ours.

E. Dynamic Environmental Experiments

As shown in Fig. 15, a dynamic environment is built to further evaluate the performance of the ACCMPS when encountering moving objects. In this environment, a whiteboard is on the right, in front of the UAV, while an obstacle that can move is on the left. As can be seen in scenario 1, ACCMPS builds an accurate map, and the planning module generates a smooth path through the gap between two objects. When the

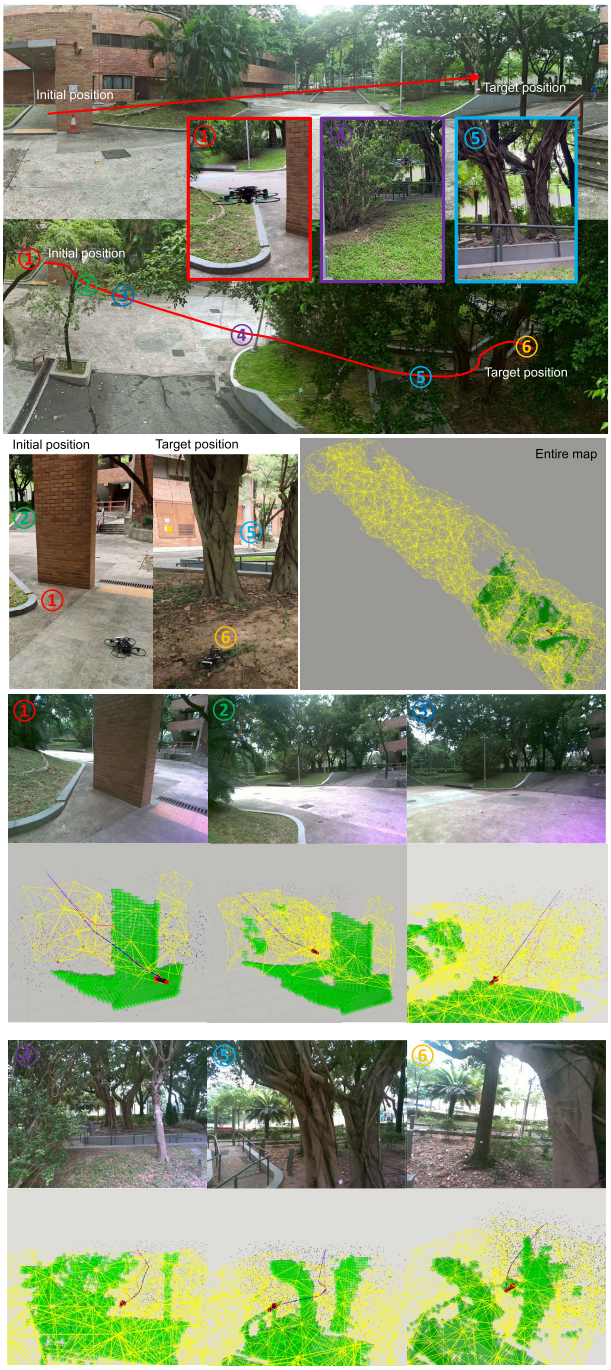


Fig. 14. Evaluation of ACCMPS in a large environment. The legend is same as Fig. 9.

UAV flies, we move the obstacle to narrow the accessway. In the case shown in scenario 2, the accessway is not big enough for the UAV to pass safely, so ACCMPS plans the path to fly over the obstacle. And finally, the UAV flies over the obstacle and plans a path going down, approaching the destination in scenario 3. In ACCMPS, dynamic environments do not incur additional visible time consumption compared to static ones. Notably, the B-spline generation module reallocates the time for traversing each segment, ensuring that the UAV adheres to kinematic constraints throughout the entire flight.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

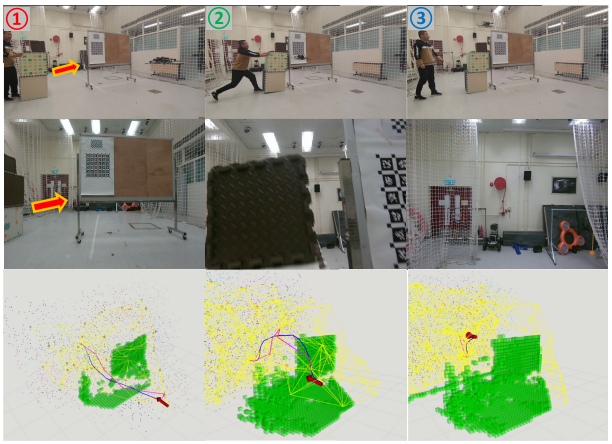


Fig. 15. Evaluation of ACCMPS in a dynamic environment. The legend is same as Fig. 9.

VI. Conclusion and Future Work

This paper presents ACCMPS, a method for efficient UAV navigation in unknown, non-convex environments. Using an adaptive space-searching algorithm, ACCMPS partitions irregular spaces into relatively convex subspaces, improving mapping and planning efficiency. It combines a sample-friendly 3D grid map for local mapping with a cluster-based weighted graph for global representation, reducing storage and computation in path searching. A particle filter further optimizes initial paths into high-quality B-spline trajectories. Real-world experiments confirm its robustness and superiority over existing algorithms in path quality, time efficiency, and storage usage when navigating challenging environments.

In future work, ACCMPS will be deployed and evaluated in large-scale outdoor environments to fully assess its performance in complex and dynamic scenarios. Current experiments demonstrate that the proposed system can update moving objects in the map and perform real-time replanning to safely avoid low-speed dynamic obstacles. However, handling high-speed dynamic obstacles requires explicit trajectory prediction to anticipate future collisions. We will extend ACCMPS with motion forecasting models and integrating predicted obstacle trajectories into the cluster-based planning framework in our future research.

References

[1] H. Huang and A. V. Savkin, "Aerial surveillance in cities: When UAVs take public transportation vehicles," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 2, pp. 1069–1080, Apr. 2023.

[2] H. Wang et al., "Structerf-SLAM: Neural implicit representation SLAM for structural environments," *Comput. Graph.*, vol. 119, Apr. 2024, Art. no. 103893.

[3] H. Wang et al., "RSS: Robust stereo SLAM with novel extraction and full exploitation of plane features," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5158–5165, Jun. 2024.

[4] Y. Zhang, Z. Zhang, and S. Wang, "Adaptive clustering quasi-line search path planning algorithm based on sampling," *IEEE Trans. Veh. Technol.*, vol. 72, no. 2, pp. 1720–1734, Feb. 2023.

[5] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM J. Optim.*, vol. 34, no. 1, pp. 507–532, Mar. 2024.

[6] Y. Yang et al., "Sampling-efficient path planning and improved actor-critic-based obstacle avoidance for autonomous robots," *Sci. China Inf. Sci.*, vol. 67, no. 5, pp. 1–18, May 2024.

[7] L. Campos-Macías, R. Aldana-López, R. de la Guardia, J. I. Parra-Vilchis, and D. Gómez-Gutiérrez, "Autonomous navigation of mavs in unknown cluttered environments," *J. Field Robot.*, vol. 38, no. 2, pp. 307–326, 2021.

[8] T. N. Canh, V.-T. Nguyen, X. HoangVan, A. Elibol, and N. Y. Chong, "S3M: Semantic segmentation sparse mapping for UAVs with RGB-D camera," in *Proc. IEEE/SICE Int. Symp. Syst. Integr. (SII)*, Jan. 2024, pp. 899–905.

[9] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "OctoMap: An efficient probabilistic 3D mapping framework based on octrees," *Autonomous Robots*, vol. 34, pp. 189–206, Feb. 2013.

[10] M. Czajka, M. Krupka, D. Kubacka, M. R. Janiszewski, and D. Belter, "A comparison of segmentation methods for semantic OctoMap generation," *Appl. Sci.*, vol. 15, no. 13, p. 7285, Jun. 2025.

[11] S. Ganesan, B. Ramalingam, and R. E. Mohan, "A hybrid sampling-based RRT path planning algorithm for autonomous mobile robot navigation," *Expert Syst. Appl.*, vol. 258, Dec. 2024, Art. no. 125206.

[12] K. Kai-shen, H. Hai-long, S. U. Zi-qi, and W. Hai-ze, "The improved informed-RRT algorithm, which optimizes the sampling strategy and integrates an artificial potential field," *J. Field Robot.*, vol. 42, no. 8, pp. 4033–4052, Dec. 2025.

[13] J. Nasir et al., "RRT-SMART: A rapid convergence implementation of RRT," *Int. J. Adv. Robotic Syst.*, vol. 10, no. 7, p. 299, Jul. 2013.

[14] E. Kozlov and T. Muslimov, "Artificial potential field method for path and trajectory planning of unmanned aerial vehicles: A review," *Unmanned Syst.*, early access, Jun. 2025.

[15] Y. Chen, "Multi-UAV avoiding barrier problem based on the improved artificial potential afield nd A-star algorithm," in *Proc. Int. Conf. Algorithms Softw. Eng. Netw. Secur.*, Apr. 2024, pp. 1–5.

[16] K. Daniel, A. Nash, S. Koenig, and A. Felner, "Theta: Any-angle path planning on grids," *J. Artif. Intell. Res.*, vol. 39, pp. 533–579, Oct. 2010.

[17] Y. Hu, D. Harabor, L. Qin, and Q. Yin, "Regarding goal bounding and jump point search," *J. Artif. Intell. Res.*, vol. 70, pp. 631–681, Feb. 2021.

[18] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "EGO-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 478–485, Apr. 2021.

[19] J. Hou et al., "Primitive-swarm: An ultra-lightweight and scalable planner for large-scale aerial swarms," *IEEE Trans. Robot.*, vol. 41, pp. 3629–3648, 2025.

[20] A. G. Wills and T. B. Schön, "Sequential Monte Carlo: A unified review," *Annu. Rev. Control, Robot., Auto. Syst.*, vol. 6, no. 1, pp. 159–182, May 2023.

[21] H. Prautzsch, W. Boehm, and M. Paluszny, *Bézier and B-Spline Techniques*, vol. 6. Cham, Switzerland: Springer, 2002.

[22] S. Liu et al., "Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-D complex environments," *IEEE Robot. Autom. Lett.*, vol. 2, no. 3, pp. 1688–1695, Jul. 2017.

[23] S. Jing, X. Guangtong, Z. Wang, T. Long, and S. Jingliang, "Safe flight corridor constrained sequential convex programming for efficient trajectory generation of fixed-wing UAVs," *Chin. J. Aeronaut.*, vol. 38, no. 1, Jan. 2025, Art. no. 103174.

[24] C. Li et al., "Marden-based homotopic enclosed safe motion corridor generation for UAV navigation in complex environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 17486–17500, 2025.

[25] J. Du, B. Huang, and B. Jia, "An efficient UAV coverage path planning method for 3-D structures," *IEEE Internet Things J.*, vol. 12, no. 15, pp. 31869–31880, Aug. 2025.

[26] Y. Cao, X. Cheng, and J. Mu, "Concentrated coverage path planning algorithm of UAV formation for aerial photography," *IEEE Sensors J.*, vol. 22, no. 11, pp. 11098–11111, Jun. 2022.

[27] L. Wang, W. Huang, H. Li, W. Li, J. Chen, and W. Wu, "A review of collaborative trajectory planning for multiple unmanned aerial vehicles," *Processes*, vol. 12, no. 6, p. 1272, 2024.

[28] X. Xing, Z. Zhou, Y. Li, B. Xiao, and Y. Xun, "Multi-UAV adaptive cooperative formation trajectory planning based on an improved MATD3 algorithm of deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 12484–12499, Sep. 2024.

[29] H. He, W. Yuan, S. Chen, X. Jiang, F. Yang, and J. Yang, "Deep reinforcement learning-based distributed 3D UAV trajectory design," *IEEE Trans. Commun.*, vol. 72, no. 6, pp. 3736–3751, Jun. 2024.

[30] B. Li, J. Wang, C. Song, Z. Yang, K. Wan, and Q. Zhang, "Multi-UAV roundup strategy method based on deep reinforcement learning CEL-MADDPG algorithm," *Expert Syst. Appl.*, vol. 245, Jul. 2024, Art. no. 123018.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.

[31] A. Jasour, W. Han, and B. C. Williams, "Convex risk-bounded continuous-time trajectory planning and tube design in uncertain non-convex environments," *Int. J. Robot. Res.*, vol. 42, no. 10, pp. 705–728, Sep. 2023.
[32] W. Han, A. Jasour, and B. Williams, "Non-Gaussian risk bounded trajectory optimization for stochastic nonlinear systems in uncertain environments," in *Proc. Int. Conf. Robot. Autom. (ICRA)*, May 2022, pp. 11044–11050.
[33] F. Meng, L. Chen, H. Ma, J. Wang, and M. Q.-H. Meng, "Learning-based risk-bounded path planning under environmental uncertainty," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 4460–4470, Jul. 2024.
[34] T. Qin, P. Li, and S. Shen, "VINS-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
[35] G. Miraglia and L. R. Hook, "Feedback motion plan verification for vehicles with bounded curvature constraints," in *Proc. IEEE 10th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON)*, Oct. 2019, pp. 0844–0854.
[36] M. Sniedovich, "Dijkstra's algorithm revisited: The dynamic programming connexion," *Control Cybern.*, vol. 35, no. 3, pp. 599–620, 2006.
[37] M. J. D. Powell, "A new algorithm for unconstrained optimization," in *Nonlinear Programming*. Amsterdam, The Netherlands: Elsevier, 1970, pp. 31–65.
[38] X. Ma, R. Gong, Y. Tan, H. Mei, and C. Li, "Path planning of mobile robot based on improved PRM based on cubic spline," *Wireless Commun. Mobile Comput.*, vol. 2022, no. 1, Jan. 2022, Art. no. 1632698.



Yifei Zhang received the B.Eng. and M.Eng. degrees in electronic and information engineering from Southwest University, Chongqing, China, in 2021 and 2024. He is currently pursuing the Ph.D. degree with the Department of Aeronautical and Aviation Engineering, The Hong Kong Polytechnic University, Hong Kong, under the supervision of Prof. Chih-Yung Wen. His research interests include mapping, path planning, simultaneous localization and mapping (SLAM), and robot navigation of UAV and UGV.



Zheng Tan is currently pursuing the Ph.D. degree with the Department of Aeronautical and Aviation Engineering, The Hong Kong Polytechnic University, Hong Kong. His research interests include information fusion for robotic systems and UAV control and navigation.



Kang Liu (Member, IEEE) received the Ph.D. degree in control science and engineering from the University of Science and Technology of China, Hefei, China, in 2022. He is currently a Research Assistant Professor with the Department of Aeronautical and Aviation Engineering (AAE), The Hong Kong Polytechnic University (HKPolyU), Hong Kong. Before becoming a Research Assistant Professor, he was a Research Associate with the Department of Computer Science, The University of Sheffield, U.K., and subsequently as a Research Post-Doctoral Fellow with AAE, HKPolyU. He is a member of IES and CAA, and serves on multiple IEEE/RAS Technical Committees.



Yefeng Yang received the Ph.D. degree from the Department of Aeronautical and Aviation Engineering (AAE), The Hong Kong Polytechnic University (HKPolyU), in 2025. He is currently a Post-Doctoral Researcher with AAE, HKPolyU. His research interests include the application and theory of reinforcement learning control, multi-agent control, and predefined-time control.



Wenyu Yang received the Ph.D. degree from The Hong Kong Polytechnic University (HKPolyU), Hong Kong, China, in February 2026. He is currently a Post-Doctoral Fellow with the Department of Aeronautical and Aviation Engineering, HKPolyU.



Shiyuan Wang (Senior Member, IEEE) received the B.Eng. and M.Eng. degrees in electronic and information engineering from Southwest Normal University, Chongqing, China, in 2002 and 2005, respectively, and the Ph.D. degree in circuit and systems from Chongqing University, Chongqing, in 2011. From August 2012 to August 2013, he was a Research Associate with The Hong Kong Polytechnic University. He is currently a Professor with the College of Electronic and Information Engineering, Southwest University, Chongqing. His research interests include adaptive signal processing, nonlinear dynamics, and simultaneous localization and mapping (SLAM).



Chih-Yung Wen received the B.S. degree from the Department of Mechanical Engineering, National Taiwan University, in 1986, and the M.S. and Ph.D. degrees from the Department of Aeronautics, California Institute of Technology, in 1989 and 1994, respectively. In 2012, he joined the Department of Mechanical Engineering, The Hong Kong Polytechnic University (HKPolyU), as a Professor. From 2015 to 2019, he was the Associate Head/Researcher with the Department of Mechanical Engineering, HKPolyU. He is currently the Chair Professor of Aeronautical Engineering with the Department of Aeronautical and Aviation Engineering and the Associate Director of the Research Institute for Sports Science and Technology, HKPolyU. In 2019, he became the Interim Head of the Department of Aeronautical and Aviation Engineering, HKPolyU. He is actively involved in professional academic activities related to mechanical and aerospace engineering at both domestic and international levels.

Authorized licensed use limited to: UNIVERSIDADE DE SAO PAULO. Downloaded on June 02,2026 at 18:48:43 UTC from IEEE Xplore. Restrictions apply.